

Guía de Teoría: Aseguramiento de la Información (RA6 - Backups y Transferencia en Entornos Reales)

Departamento de Informática - ASIR

April 14, 2026

Introducción

Esta guía detalla las tareas críticas de un administrador de sistemas para garantizar la supervivencia de los datos. No solo aprenderemos los comandos básicos, sino también las estrategias de nivel empresarial para gestionar copias de seguridad en entornos de producción masivos.

1 Entornos de Trabajo: Desarrollo, Test y Producción

- **Desarrollo (Dev):** Donde se crea el código. Se permiten reinicios y pérdidas de datos.
- **Test / Staging:** Réplica de producción para validar restauraciones. Si un backup funciona aquí, funcionará en el mundo real.
- **Producción (Prod):** El entorno crítico. Aquí los fallos cuestan dinero y los backups son automáticos.

2 Conceptos Fundamentales de Recuperación

2.1 Tipos de Copia: El peso del cambio

- **Backup Completo (Full):** Fotocopia total de la BD. Es la base de cualquier restauración.
- **Backup Incremental:** Guarda solo lo que cambió desde el **último** backup. Es rápido pero genera muchos archivos.
- **Backup Diferencial:** Guarda lo que cambió desde el último **Completo**.

Pregunta de reflexión

¿Qué ventajas e inconvenientes tiene cada tipo de sistema de restauración?

2.2 El Log Binario y la Rotación de Logs

El **Log Binario (Binary Log)** es la "caja negra" del servidor. Registra cada INSERT, UPDATE o DELETE en orden cronológico. Para poder copiar estos archivos sin riesgo, usamos la **Rotación de Logs**.

```
$ mysql -u admin -p -e "FLUSH LOGS;"
```

3 Copias de Seguridad: Herramientas CLI

¡Ojo!

mysqldump se ejecuta en la **Terminal de Linux (\$)**, **NUNCA** dentro de MySQL (mysql>).

3.1 Nivel 1: Uso Muy Básico

```
$ mysqldump -u admin -p logistica_global > copia_full.sql
```

3.2 Nivel 2: Uso Básico (Filtrado)

- Solo una tabla:

```
$ mysqldump -u admin -p logistica_global almacenes > solo_almacenes.sql
```

- Solo Esquema (Sin datos):

```
$ mysqldump -u admin -p --no-data logistica_global > estructura.sql
```

- Solo Datos (Sin estructura):

```
$ mysqldump -u admin -p --no-create-info logistica_global > solo_datos.sql
```

3.3 Nivel 3: Uso Avanzado (Producción)

- El GPS del Backup (–source-data=2):

```
$ mysqldump -u admin -p --source-data=2 logistica_global > lunes.sql
```

¿Por qué es fundamental este parámetro?

Cuando haces un backup completo, el servidor sigue recibiendo datos. ¿Cómo sabes en qué archivo de log binario debes empezar a buscar los cambios posteriores? `--source-data=2` escribe una línea de comentario al principio del archivo SQL:

```
-- CHANGE REPLICATION SOURCE TO MASTER_LOG_FILE='binlog.000003',  
MASTER_LOG_POS=154;
```

Valores posibles:

- =1: Escribe el comando listo para ejecutarse (se usa para configurar servidores esclavos/réplicas).
- =2: Escribe el comando como un **comentario SQL** (prefijo `--`). Es el estándar para backups porque nos da la información sin ejecutar nada por error.

- **Evitar bloqueos en Producción:**

```
$ mysqldump -u admin -p --single-transaction logistica_global > backup.sql
```

¿Cómo funciona `--single-transaction`?

Normalmente, para obtener un respaldo "coherente" (donde los datos no cambien a mitad del proceso), las herramientas tendrían que bloquear las tablas para que nadie escriba en ellas.

`--single-transaction` evita esto al iniciar una transacción con aislamiento de lectura repetible (*Repeatable Read*). Esto permite que `mysqldump` vea una "fotografía" estática de los datos en el momento exacto en que comenzó el comando, mientras que el resto de los usuarios pueden seguir insertando o modificando datos sin enterarse.

Funcionamiento interno:

1. Envía una instrucción `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ`.
2. Ejecuta un `START TRANSACTION`.
3. A partir de ahí, **InnoDB** utiliza su sistema de control de versiones (**MVCC**) para asegurar que todas las consultas lean los datos tal como estaban en ese instante.

Estrategias en MySQL: Cómo crear un Diferencial REAL

Dado que `mysqldump` solo hace volcados completos, para crear un backup diferencial real en `mysql` (solo los cambios desde el último Full) debemos usar `mysqlbinlog`.

4 Manual de Recuperación Incremental (mysqlbinlog)

Cuando el servidor falla, el backup del lunes no es suficiente; faltan los cambios posteriores. Esos cambios están en los **Logs Binarios**.

4.1 Gestión de Logs: Creación y Visualización

Antes de recuperar, debemos saber qué logs tenemos y cómo "forzar" que se guarden.

- **¿Está activo el log binario?**
`mysql> SHOW VARIABLES LIKE 'log_bin';`
- **Listar archivos de log disponibles:**
`mysql> SHOW BINARY LOGS;`
Muestra todos los archivos generados y su tamaño en bytes.
- **Ver el log que se está usando AHORA:**
`mysql> SHOW MASTER STATUS;`
Indica en qué archivo y posición exacta se escriben los datos en este instante.
- **Generar un log hasta el momento actual:**
`mysql> FLUSH LOGS;` *Cierra el log binario y comienza uno nuevo para poder trabajar con el anterior.*

Ejecutar comandos sql desde bash

Se pueden enviar comandos de sql de forma similar a como cargamos las bases de datos. Es habitual, en los scripts de aseguramiento de la información, ejecutar:

```
$ mysql -u admin -p -e "FLUSH LOGS;"
```

La importancia del FLUSH LOGS

Cuando queremos realizar un backup incremental de los cambios de hoy, no podemos simplemente copiar el archivo que el servidor tiene abierto (porque sigue cambiando). Al ejecutar `FLUSH LOGS;`, el servidor cierra el archivo actual y abre uno nuevo. El archivo cerrado se vuelve "estático" y ya podemos moverlo a nuestra carpeta de backups con seguridad.

4.2 Acceso a Logs sin privilegios (SUDO)

Por defecto, los archivos binlog en `/var/lib/mysql` solo son legibles por el usuario `root` y el grupo `mysql`. Para trabajar, tenemos, principalmente, dos opciones:

- **Opción A: Lectura Remota (Recomendada)**
Podemos pedirle al servidor MySQL que nos "envíe" el contenido del log a través de la red local (usando el protocolo de replicación). No necesitamos acceso directo a la carpeta de datos.

```
$ mysqlbinlog --read-from-remote-server  
-u admin -p -h localhost binlog.000001
```

*Nota: date cuenta cómo este comando puede funcionar en servidores remotos. Solamente será necesario indicar la IP como argumento posterior a la **h** de **host**.*

- **Opción B: Permisos de Sistema**

Añadir nuestro usuario de Linux al grupo `mysql`. Requiere cerrar y abrir sesión para que los cambios surtan efecto.

```
$ sudo usermod -aG mysql $USER
```

4.3 Ver un log binario

```
$ mysqlbinlog binlog.000001
```

Si el servidor utiliza el formato de log basado en filas (ROW), el comando anterior te mostrará código en Base64 que es imposible de leer. Para "traducirlo" a sentencias SQL legibles (como `INSERT`, `UPDATE`, `DELETE`), debes usar los modificadores de verbose:

```
$ mysqlbinlog --base64-output=DECODE-ROWS -v binlog.000001
```

4.4 El flujo de restauración (The Pipe)

Restaurar un log binario no es "importar un archivo", es **re-ejecutar la historia**. Como el archivo binario no es SQL plano, necesitamos un traductor (`mysqlbinlog`) que lea los eventos y los convierta en comandos que el servidor entienda.

```
$ mysqlbinlog binlog.000001 | mysql -u admin -p
```

¿Qué ocurre aquí?

1. `mysqlbinlog` abre el archivo y empieza a "leer" la cronología.
2. La tubería (`|`) envía cada comando traducido directamente al cliente `mysql`.
3. El cliente `mysql` ejecuta esos comandos en el servidor en tiempo real.

4.5 Point-in-Time Recovery (PITR): Cirugía de Datos

El PITR es la capacidad de volver a un momento exacto en el tiempo, justo un segundo antes de que ocurriera un desastre (como un `DROP DATABASE` accidental).

- **Filtrado Temporal (`--stop-datetime`):** Es el método "humano". Si sabemos que el becario borró la tabla a las 10:05 AM, le decimos a `mysqlbinlog` que procese todo lo que ocurrió ****hasta las 10:04:59****.

```
$ mysqlbinlog --stop-datetime="2026-04-17  
10:04:59" binlog.000004 | mysql...
```

Limitación: Si ocurren mil transacciones en el mismo segundo, no podemos ser quirúrgicos.

- **Identificación por Posición (`--stop-position`):** Es el método "máquina" y el más preciso. Cada evento en el log tiene un número entero único llamado **End_log_pos**. Si buscamos en el log (usando `-v`) y vemos que el comando del desastre empieza en la posición 1540, le decimos al sistema que se detenga exactamente en la 1540.

```
$ mysqlbinlog --stop-position=1540 binlog.000004 | mysql...
```

Ventaja: Es imposible equivocarse de evento. Restauramos hasta el bit anterior al error.

4.6 El eslabón perdido: ¿Dónde empezar? (`--start-position`)

Saber dónde parar (`--stop`) es inútil si no sabemos dónde empezar. Por este motivo, hay que ser ordenado con cuándo comienza cada log binario.

No obstante, si aplicamos un log binario completo desde el principio sobre una base de datos recién restaurada con `mysqldump`, el proceso fallará por **conflictos de clave primaria**. Estaríamos intentando re-insertar datos que ya vienen en el volcado completo.

Por ese motivo, los logs binarios tienen la posición. Algo como `-- MASTER_LOG_POS=154;`

Esa puede usarse de "posición de salida". Para que la restauración sea coherente y no haya duplicados, debemos empezar el `mysqlbinlog` exactamente en ese número.

```
$ mysqlbinlog --start-position=154  
--stop-position=1234 binlog.000001 | mysql ...
```

*Nota: De esta forma, solo aplicamos los cambios ocurridos **después** del backup, garantizando una restauración limpia y sin errores.*

5 Simulación de Desastre: Una Semana en Empresa Segura

5.1 Día 1: El Punto de Partida (Lunes)

1. **Paso 1: Inicialización.** Carga el entorno base (4 empleados). Es el archivo `src/00_backup_setup.sql`
2. **Paso 2: Backup Completo (BASE).**

```
$ mysqldump -u admin -p --source-data=2  
empresa_segura > lunes_completo.sql
```

3. **Paso 3: Verificación.** Comprueba que el archivo SQL existe y tiene datos.

5.2 Día 2: Actividad y Backup Incremental (Martes)

1. **Paso 4: Alta de datos.** Inserta a 'Roberto' en 'Ventas'.
2. **Paso 5: Rotación de Logs.**

```
$ mysql -u admin -p -e "FLUSH LOGS;"
```

3. **Paso 6: Identificación.** Localiza log binario correspondiente `binlog.??????`.
4. **Paso 7: Backup Incremental 1.** Guarda ese archivo a buen recaudo.

Puedes guardar una hoja de cálculo con los ficheros y la información que contienen.

¿Cuándo comienza ese log?

Antes de seguir, plantéate, ¿cuándo comienza ese log binario? ¿Tal vez haya que utilizar `--start-position`? ¿Cómo podríamos haberlo hecho sin `--start-position`?

5.3 Día 3: El Backup Diferencial (Miércoles)

1. **Paso 8: Alta de datos.** Inserta a 'Marta' en 'IT' y vuelve a ejecutar `FLUSH LOGS;`.
2. **Paso 9: Estrategia Diferencial REAL.** Unimos los binlogs desde el Lunes:

```
$ mysqlbinlog binlog.000001 binlog.000002 > miercoles_diferencial.sql
```

Consiste en unir todos los Logs Binarios generados desde el Lunes en un único archivo SQL. Esto ocupa mucho menos que un volcado total y nos permite restaurar rápidamente la base inicial del Lunes a su estado del Miércoles sin procesar archivos sueltos.

Nota: Realizar un `mysqldump` a mitad de semana NO es un diferencial, es simplemente otro backup completo. La única forma de ahorrar espacio y tiempo es mediante la manipulación de los Logs Binarios.

5.4 Día 4: Más cambios (Jueves)

1. **Paso 10: Modificación.** Sube el salario de 'Ana García' a 40000€.
2. **Paso 11: Alta de datos.** Inserta a 'Julian' en 'Recursos Humanos'.
3. **Paso 12: Backup Incremental 2.** Ejecuta `FLUSH LOGS;` y guarda el binlog.000003.

5.5 Día 5: EL DESASTRE (Viernes)

1. **Paso 13: Error Humano (10:00 AM).** El becario borra la BD:

```
$ mysql -u admin -p -e "DROP DATABASE empresa_segura;"
```

2. **Paso 14: Confirmación.** Verifica que los datos han desaparecido.
3. **Paso 15: Análisis Forense.** Revisa el `log_error` para ver la hora exacta.

5.6 Día 6: La Gran Restauración (Sábado)

1. **Paso 16: Cimientos.** Restaura el Backup Completo del Lunes:

```
$ mysql -u admin -p < lunes_completo.sql
```

2. **Paso 17: Salto Diferencial.** Aplica el diferencial del Miércoles:

```
$ mysql -u admin -p empresa_segura < miercoles_diferencial.sql
```

3. **Paso 18: Recuperación Incremental.** Aplica los cambios del Jueves:

```
$ mysqlbinlog binlog.000003 | mysql -u admin -p empresa_segura
```

4. **Paso 19: PITR.** Recupera el Viernes evitando el borrado:

```
$ mysqlbinlog --stop-datetime="2026-04-17  
09:59:00" binlog.000004 | mysql...
```

*Ojo, tendrás que utilizar la fecha y la hora reales del desastre o la posición del error.
Investiga el estándar actual: **GTID***

5.7 Día 7: Auditoría y Cierre (Domingo)

1. **Paso 20: Verificación Final.** Comprueba que los 7 empleados están presentes.

6 Solucionario y Respuestas

Resultados de la Simulación

Al finalizar el Paso 20, la tabla `empleados` debe tener 7 registros. El RTO se ha minimizado usando el diferencial del miércoles (generado con `mysqlbinlog`), evitando restaurar logs binarios sueltos de los primeros días.